

✓ Project Title

E-Commerce Sales Analytics - Comparative Analysis of Amazon and Flipkart Sales

Domain

E-Commerce and Retail Analytics

Objective

- Analyze and compare sales performance between Amazon and Flipkart platforms.
 - Understand product category performance and identify top revenue-generating products.
 - Analyze customer ratings and purchasing behavior.
 - Study payment method preferences and city-wise sales trends.
 - Identify patterns in sales data using Exploratory Data Analysis (EDA).
 - Develop visualizations to generate business insights and support data-driven decision-making.
-

Dataset Information

Source: https://github.com/sanjoydutt/amazon_sales_2025_analysis

Timeline: Sales transaction data from 2025

Description:

- The Amazon dataset contains e-commerce transaction details including product information, customer details, pricing, quantity, payment methods, ratings, and sales information from Amazon platforms.
- The flipkart dataset contains e-commerce transaction details including product information, customer details, pricing, quantity, payment methods, ratings, and sales information from Amazon platforms.

Amazon Size:

- Rows: 15,000
- Columns: 14 Columns

Flipkart Size:

- Rows: 14,999

- Columns: 12 Columns

Columns:

- Order_ID
 - Date
 - Customer_ID
 - Product_Name
 - Product_category
 - Quantity
 - Unit_price
 - Total_Sales
 - Payment_method
 - Customer_rating
 - City
 - Country
-

Expected Outcomes

- Compare Amazon and Flipkart revenue performance.
 - Identify top-selling products and high-performing categories.
 - Analyze customer satisfaction using ratings.
 - Understand city-wise sales contribution.
 - Analyze payment method preferences.
 - Identify monthly sales trends and purchasing patterns.
 - Provide business recommendations based on data-driven insights.
-

Type of Analysis

Descriptive Analysis (Summarizing sales, customer, product, and platform performance)

Exploratory Data Analysis (EDA)

Stages for Data Analytics Project

Stage 1 – Problem Definition and Dataset Selection

Problem Definition:

E-commerce platforms generate large amounts of sales and customer data, but extracting meaningful insights from this data is challenging.

This project focuses on analyzing Amazon and Flipkart sales data to understand revenue performance, customer behavior, product trends, and purchasing patterns.

The analysis helps identify business opportunities, compare platform performance, and support strategic decision-making through data-driven insights.

```
# Import required libraries

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import matplotlib.pyplot as plt
import seaborn as sns

# Import data
from google.colab import files

uploaded = files.upload()

Amazon = pd.read_csv(next(iter(uploaded)))

print("Amazon data set loaded succesfully")

uploaded = files.upload()

flipkart = pd.read_csv(next(iter(uploaded)))

print("flipkart sales data set loaded succesfully")
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving amazon_sales_2025_INR.csv to amazon_sales_2025_INR (1).csv

Amazon data set loaded succesfully

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving flipkart dataset.csv to flipkart dataset.csv

```
Amazon.head()
```

	Order_ID	Date	Customer_ID	Product_Category	Product_Name	Quantity	Unit
0	ORD100000	25-01-2025	CUST2796	Home & Kitchen	Cookware Set	2	
1	ORD100001	28-08-2025	CUST9669	Beauty	Hair Dryer	1	
2	ORD100002	27-02-2025	CUST5808	Electronics	Tablet	3	
3	ORD100003	24-02-2025	CUST5889	Electronics	Headphones	5	
4	ORD100004	15-06-2025	CUST9005	Clothing	Saree	5	

flipkart.head()

	order_id	Product_Name	Product_category	customer_id	city	Unit_price
0	FK1001	Adidas Ultra 664	Toys	CUST1001	Hyderabad	30215.24
1	FK1002	LG Series 124	Fashion	CUST1002	Mumbai	27624.41
2	FK1003	Redmi Model 35	Beauty	CUST1003	Ahmedabad	30728.70
3	FK1004	Sony Edition	Toys	CUST1004	Delhi	23217.94

```
print("Amazon Shape:", Amazon.shape)

print("Flipkart Shape:", flipkart.shape)
```

Amazon Shape: (15000, 14)
Flipkart Shape: (14999, 12)

```
Amazon.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              15000 non-null  object
1   Date                  15000 non-null  object
```

```

2  Customer_ID      15000 non-null object
3  Product_Category 14988 non-null object
4  Product_Name     14974 non-null object
5  Quantity         15000 non-null int64
6  Unit_Price_INR   14982 non-null float64
7  Total_Sales_INR  14969 non-null float64
8  Payment_Method   14983 non-null object
9  Delivery_Status  15000 non-null object
10 Review_Rating    14987 non-null float64
11 Review_Text      15000 non-null object
12 City             14990 non-null object
13 Country           15000 non-null object
dtypes: float64(3), int64(1), object(10)
memory usage: 1.6+ MB

```

```
flipkart.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              14999 non-null object
1   Product_Name          14999 non-null object
2   Product_category      14987 non-null object
3   customer_id           14999 non-null object
4   city                  14999 non-null object
5   Unit_price            14999 non-null float64
6   Order_Date            14999 non-null object
7   Payment_method        14983 non-null object
8   customer_rating       14982 non-null float64
9   Total_Sales           14984 non-null float64
10  Country               14986 non-null object
11  Quantity              14987 non-null float64
dtypes: float64(4), object(8)
memory usage: 1.4+ MB

```

```
Amazon.describe()
```

	Quantity	Unit_Price_INR	Total_Sales_INR	Review_Rating
count	15000.000000	14982.000000	14969.000000	14987.000000
mean	2.984667	24954.488037	74550.971152	3.039367
std	1.422826	14403.149615	59393.929551	1.411114
min	1.000000	202.570000	204.050000	1.000000
25%	2.000000	12501.582500	27053.700000	2.000000
50%	3.000000	24878.755000	57273.550000	3.000000
75%	4.000000	37496.017500	112199.940000	4.000000
max	5.000000	49994.430000	249955.500000	5.000000

```
flipkart.describe()
```

	Unit_price	customer_rating	Total_Sales	Quantity
count	14999.000000	14982.000000	14984.000000	14987.000000
mean	23717.129293	4.004767	70441.006491	2.984787
std	14764.362443	0.576001	58741.390014	1.422835
min	108.190000	3.000000	173.580000	1.000000
25%	11276.830000	3.510000	24304.370000	2.000000
50%	22580.780000	4.010000	52594.900000	3.000000
75%	34496.365000	4.510000	103827.060000	4.000000
max	59995.800000	5.000000	299632.900000	5.000000

```
Amazon.isnull().sum()
```

	0
Order_ID	0
Date	0
Customer_ID	0
Product_Category	12
Product_Name	26
Quantity	0
Unit_Price_INR	18
Total_Sales_INR	31
Payment_Method	17
Delivery_Status	0
Review_Rating	13
Review_Text	0
City	10
Country	0

dtype: int64

```
flipkart.isnull().sum()
```

	0
order_id	0
Product_Name	0
Product_category	12
customer_id	0
city	0
Unit_price	0
Order_Date	0
Payment_method	16
customer_rating	17
Total_Sales	15
Country	13
Quantity	12

dtype: int64

```
Amazon.duplicated().sum()
```

```
np.int64(0)
```

```
flipkart.duplicated().sum()
```

```
np.int64(0)
```

✓ Stage 2 – Data Cleaning and Preparation

- Handled missing values in both datasets.
- Standardized column names between Amazon and Flipkart datasets.
- Removed unnecessary columns.
- Filled missing values using appropriate methods.
- Converted data types for analysis.
- Created a clean combined dataset.

```
# Convert columns to numeric
Amazon["Quantity"] = pd.to_numeric(
    Amazon["Quantity"],
    errors="coerce"
```



```
)

Amazon["Unit_Price_INR"] = pd.to_numeric(
    Amazon["Unit_Price_INR"],
    errors="coerce"
)

Amazon["Total_Sales_INR"] = pd.to_numeric(
    Amazon["Total_Sales_INR"],
    errors="coerce"
)

Amazon["Review_Rating"] = pd.to_numeric(
    Amazon["Review_Rating"],
    errors="coerce"
)

# Fill Unit Price by product average
Amazon["Unit_Price_INR"] = Amazon.groupby(
    "Product_Name"
)["Unit_Price_INR"].transform(
    lambda x: x.fillna(x.mean())
)

# Fill remaining Unit Price with overall average
Amazon["Unit_Price_INR"] = Amazon["Unit_Price_INR"].fillna(
    Amazon["Unit_Price_INR"].mean()
)

# Fill Total Sales by product average
Amazon["Total_Sales_INR"] = Amazon.groupby(
    "Product_Name"
)["Total_Sales_INR"].transform(
    lambda x: x.fillna(x.mean())
)

# Fill remaining Total Sales with overall average
Amazon["Total_Sales_INR"] = Amazon["Total_Sales_INR"].fillna(
    Amazon["Total_Sales_INR"].mean()
)

Amazon["Review_Rating"] = Amazon["Review_Rating"].fillna(
    Amazon.groupby("Product_Name")["Review_Rating"].transform("mean")
)

# Check null values
Amazon.isnull().sum()
```

	0
Order_ID	0
Date	0
Customer_ID	0
Product_Category	0
Product_Name	0
Quantity	0
Unit_Price_INR	0
Total_Sales_INR	0
Payment_Method	0
Delivery_Status	0
Review_Rating	0
Review_Text	0
City	0
Country	0

dtype: int64

```

Amazon["Product_Name"] = Amazon["Product_Name"].fillna("Unknown")

Amazon["City"] = Amazon["City"].fillna("Unknown")

Amazon["Payment_Method"] = Amazon["Payment_Method"].fillna("Unknown")

# Create product-category mapping from existing values

product_category_map = (
    Amazon.dropna(subset=["Product_Category"])
    .groupby("Product_Name")["Product_Category"]
    .first()
)

# Fill missing category using product name

Amazon["Product_Category"] = Amazon.apply(
    lambda row: product_category_map[row["Product_Name"]]
    if pd.isna(row["Product_Category"])
    and row["Product_Name"] in product_category_map
    else row["Product_Category"],
    axis=1

```

```
)
```

```
Amazon.isnull().sum()
```

[Show hidden output](#)

```
# =====  
# FLIPKART DATA CLEANING  
# =====  
  
# 1. Check initial null values  
  
flipkart.isnull().sum()  
  
# =====  
# 2. Fix data types  
# =====  
  
import pandas as pd  
import numpy as np  
  
flipkart["Quantity"] = pd.to_numeric(  
    flipkart["Quantity"],  
    errors="coerce"  
)  
  
flipkart["Unit_price"] = pd.to_numeric(  
    flipkart["Unit_price"],  
    errors="coerce"  
)  
  
flipkart["Total_Sales"] = pd.to_numeric(  
    flipkart["Total_Sales"],  
    errors="coerce"  
)  
  
flipkart["customer_rating"] = pd.to_numeric(  
    flipkart["customer_rating"],  
    errors="coerce")  
  
# =====  
# 4. Fill Quantity using Total_Sales / Unit_price  
# =====  
  
flipkart.loc[  
    flipkart["Quantity"].isnull(),  
    "Quantity"
```

```

] = (
    flipkart["Total_Sales"] /
    flipkart["Unit_price"]
)

flipkart["Quantity"] = flipkart["Quantity"].fillna(
    flipkart["Quantity"].median()
)

# Round quantity

flipkart["Quantity"] = flipkart["Quantity"].round()

# =====
# 3. Fill Total_Sales using Quantity * Unit_price
# =====

flipkart.loc[
    flipkart["Total_Sales"].isnull(),
    "Total_Sales"
] = (
    flipkart["Quantity"] *
    flipkart["Unit_price"]
)

# =====
# 5. Fill Product Category
# using Product Name mapping
# =====

category_map = (
    flipkart
    .dropna(subset=["Product_category"])
    .groupby("Product_Name")["Product_category"]
    .first()
)

flipkart["Product_category"] = flipkart.apply(
    lambda row:
        category_map[row["Product_Name"]]
        if pd.isna(row["Product_category"])
        and row["Product_Name"] in category_map
        else row["Product_category"],
    axis=1
)

```

```
# Remaining category missing

flipkart["Product_category"] = (
    flipkart["Product_category"]
    .fillna("Unknown")
)

# =====
# 6. Fill categorical columns
# =====

flipkart["Payment_method"] = (
    flipkart["Payment_method"]
    .fillna("Unknown")
)

flipkart["Country"] = (
    flipkart["Country"]
    .fillna("Unknown")
)

# =====
# 7. Fill customer rating
# =====

flipkart["customer_rating"] = (
    flipkart["customer_rating"]
    .fillna(
        flipkart["customer_rating"].median()
    )
)

# =====
# 8. Remove duplicates
# =====

flipkart = flipkart.drop_duplicates()

# =====
```

```
# 9. Final null check
# =====

print("Final Null Values")

print(flipkart.isnull().sum())

# =====
# 10. Check final shape
# =====

print("Final Shape:", flipkart.shape)
```

[Show hidden output](#)

```
# Adding Platform column

Amazon["Platform"] = "Amazon"
flipkart["Platform"] = "Flipkart"

Amazon.head()
flipkart.head()
```

[Show hidden output](#)

✓ Stage 3 – Data Preparation and Exploratory Data Analysis

Performed the following steps:

- Created the final combined dataset by merging Amazon and Flipkart cleaned datasets.
- Ensured all required columns were standardized for further analysis.
- Verified the final dataset structure, missing values, and data consistency.
- Performed Exploratory Data Analysis (EDA) to understand sales and customer patterns.
- created visualisations

Analysis performed on:

- Platform-wise sales comparison
- Product category performance
- Top-selling products
- Customer rating analysis

- Payment method analysis
- City-wise sales analysis
- Monthly sales trends

```
Amazon.columns = Amazon.columns.str.strip()

Amazon = Amazon.rename(columns={
    "Order_ID": "order_id",
    "Product_Name": "Product_Name",
    "Product_Category": "Product_category",
    "Product_Name": "Product_Name",
    "Quantity": "Quantity",
    "Unit_Price_INR": "Unit_price",
    "Total_Sales_INR": "Total_Sales",
    "Payment_Method": "Payment_method",
    "Review_Rating": "customer_rating",
    "Country": "Country",
    "Date": "Order_Date",
    "Platform": "Platform",
    "City": "city",
    "Customer_ID": "customer_id",
})

print(Amazon.columns)
print(flipkart.columns)
```

[Show hidden output](#)

```
common_columns = Amazon.columns.intersection(flipkart.columns)

common_columns

Amazon_common = Amazon[common_columns]

flipkart_common = flipkart[common_columns]

final_data = pd.concat(
    [Amazon_common, flipkart_common],
    ignore_index=True
)
```

```
final_data.to_csv(
    "Amazon_vs_Flipkart_Final.csv",
    index=False
)
import os
```

```
os.listdir()

print(os.path.abspath("Amazon_vs_Flipkart_Final.csv"))

print(os.path.exists("/content/Amazon_vs_Flipkart_Final.csv"))

files.download("/content/Amazon_vs_Flipkart_Final.csv")
```

[Show hidden output](#)

```
final_data.shape

final_data.columns

final_data.info()
```

[Show hidden output](#)

```
final_data.describe()
```

[Show hidden output](#)

```
# Convert Unknown to NaN and make numeric
final_data["customer_rating"] = pd.to_numeric(
    final_data["customer_rating"],
    errors="coerce"
)

# Find average rating
avg_rating = final_data["customer_rating"].mean()

print("Average Rating:", avg_rating)

# Fill missing ratings with average
final_data["customer_rating"] = final_data["customer_rating"].fillna(
    avg_rating
)

[col for col in final_data.columns if "date" in col.lower()]

final_data.isnull().sum()
```

[Show hidden output](#)

```
# Data Visualization

# Bar Chart - Total Sales Comparison (Amazon vs Flipkart)
```



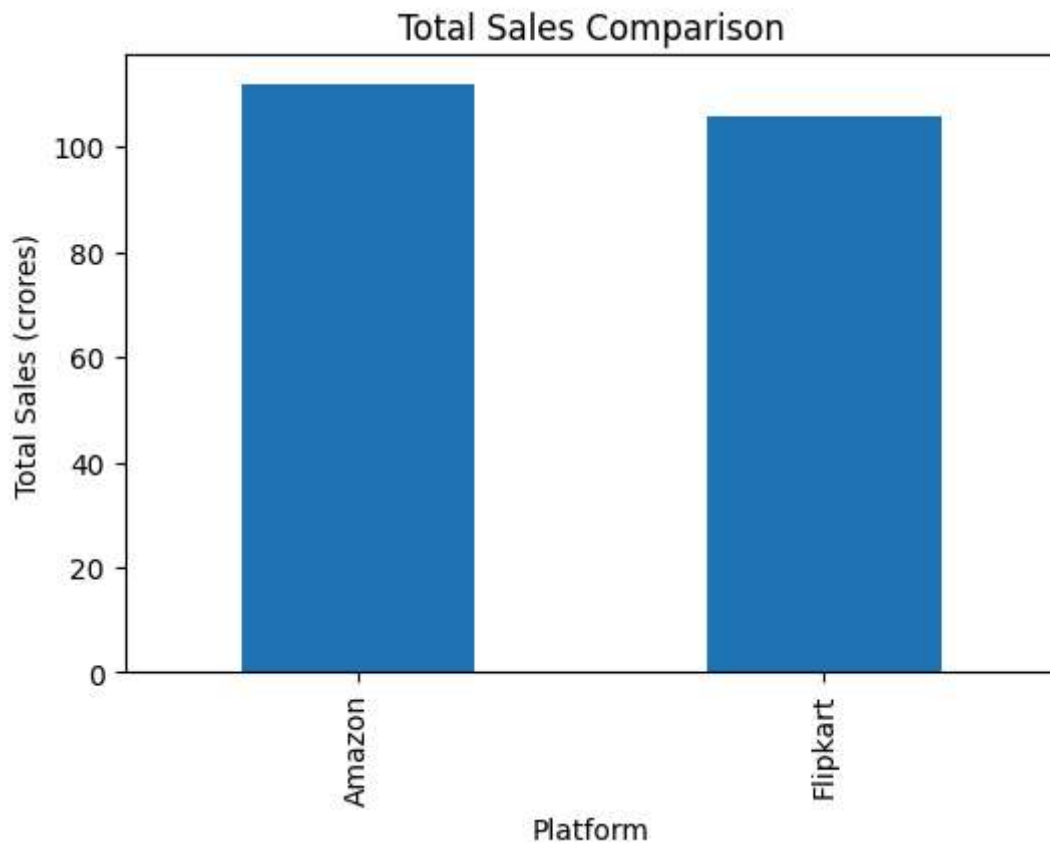
```
platform_sales = final_data.groupby(
    "Platform"
)["Total_Sales"].sum()/ 10000000

plt.figure(figsize=(6,4))

platform_sales.plot(kind="bar")

plt.title("Total Sales Comparison")
plt.xlabel("Platform")
plt.ylabel("Total Sales (crores)")

plt.show()
```



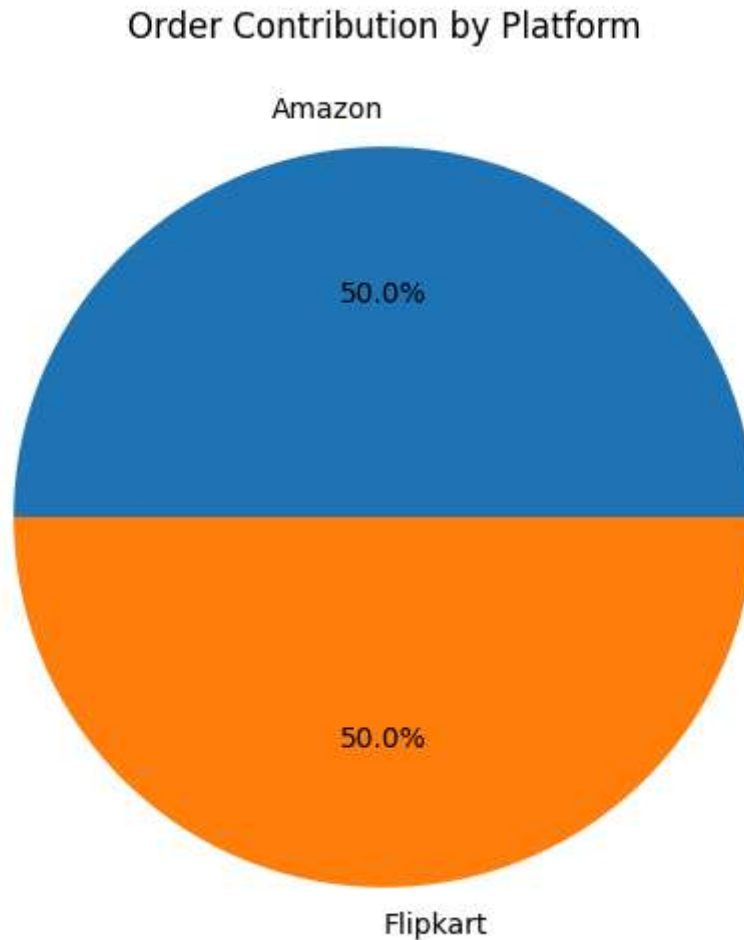
```
# Pie Chart – Order Share by Platform

orders = final_data["Platform"].value_counts()

plt.figure(figsize=(6,6))
orders.plot(
    kind="pie",
    autopct="%1.1f%%"
)

plt.title("Order Contribution by Platform")
plt.ylabel("")
```

```
plt.show()
```



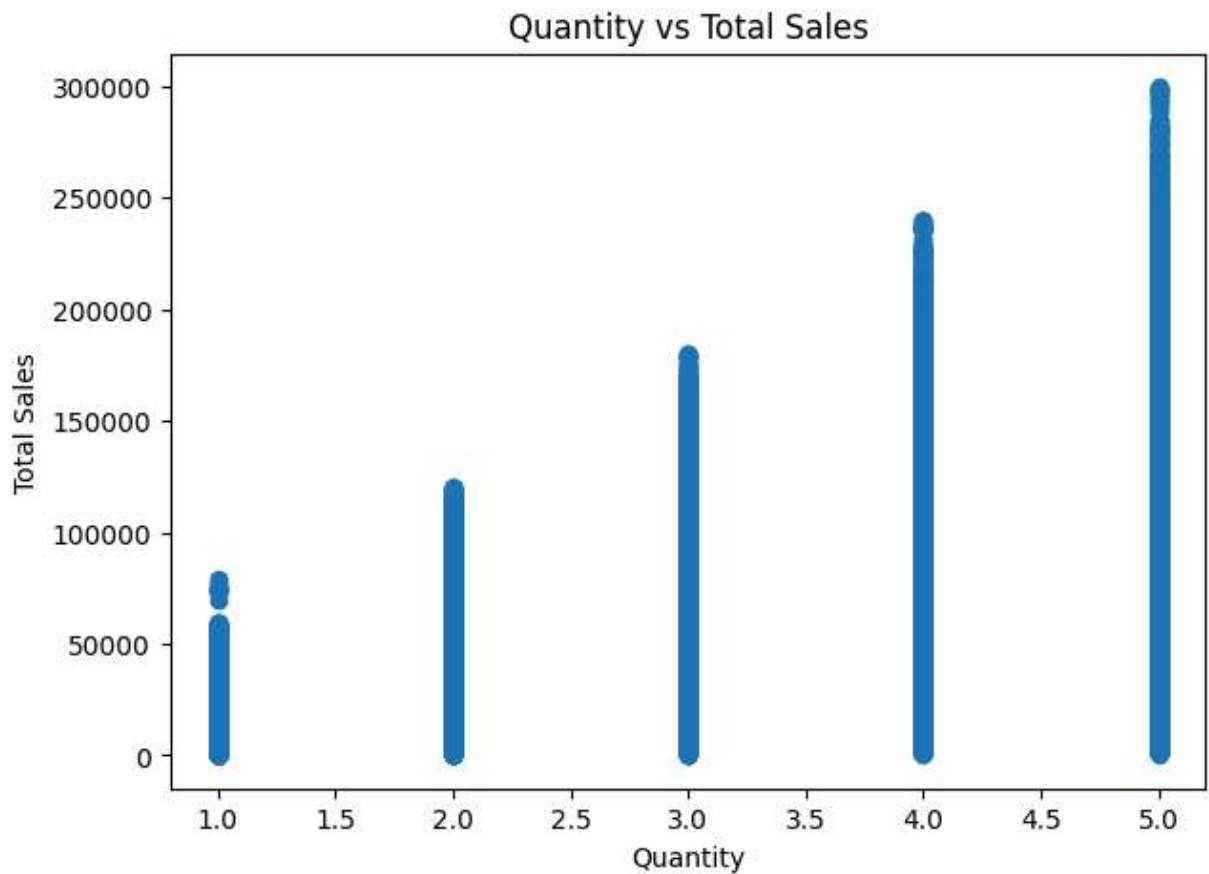
```
# Quantity Vs Total Sales

plt.figure(figsize=(7,5))

plt.scatter(
    final_data["Quantity"],
    final_data["Total_Sales"]
)

plt.title("Quantity vs Total Sales")
plt.xlabel("Quantity")
plt.ylabel("Total Sales")

plt.show()
```

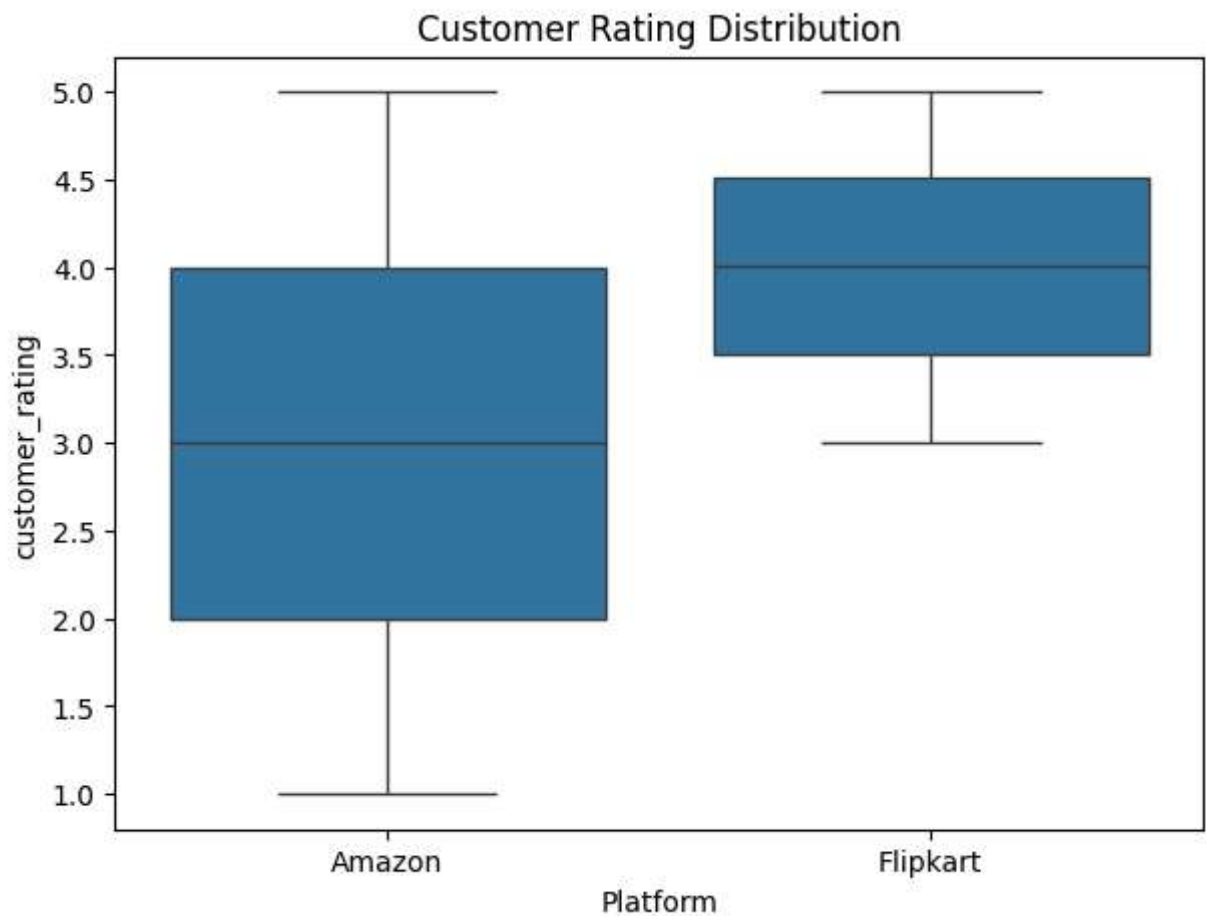


```
# Box Plot Rating distribution by Platform
plt.figure(figsize=(7,5))

sns.boxplot(
    x="Platform",
    y="customer_rating",
    data=final_data
)

plt.title("Customer Rating Distribution")

plt.show()
```



```
# Horizontal Bar Chart – Top 10 Products by Sales
```

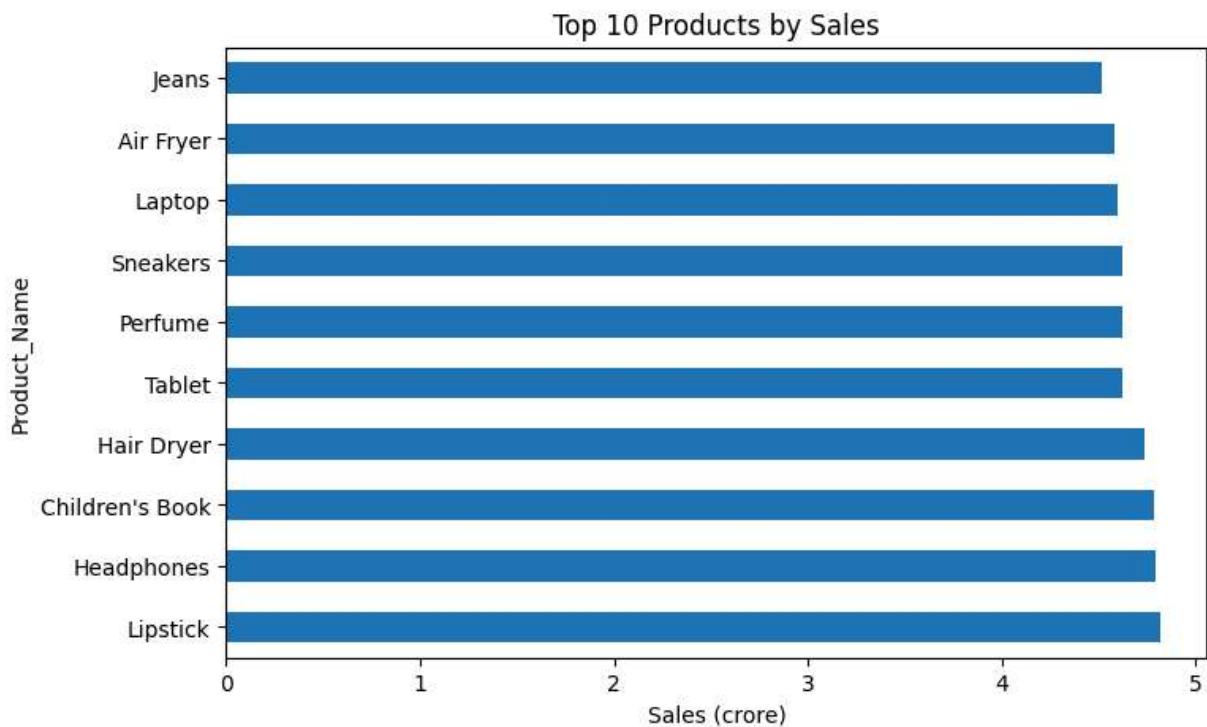
```
top_products = final_data.groupby(
    "Product_Name"
)["Total_Sales"].sum().sort_values(
    ascending=False
).head(10)/10000000

plt.figure(figsize=(8,5))

top_products.plot(
    kind="barh"
)

plt.title("Top 10 Products by Sales")
plt.xlabel("Sales (crore)")

plt.show()
```



```
# Area Chart
```

```
# Remove extra spaces from column names
```

```
final_data.columns = final_data.columns.str.strip()
```

```
# Find date column
```

```
date_col = [col for col in final_data.columns if "date" in col.lower()]
```

```
print("Date column found:", date_col)
```

```
# Convert date column
```

```
final_data[date_col[0]] = pd.to_datetime(  
    final_data[date_col[0]],  
    dayfirst=True,  
    errors="coerce"  
)
```

```
# Create Month column
```

```
final_data["Month"] = final_data[date_col[0]].dt.month_name()
```

```
# Check result
final_data[[date_col[0], "Month"]].head()

# Month order
month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

# Monthly sales
monthly_sales = final_data.groupby(
    "Month"
)["Total_Sales"].sum()

# Arrange months correctly
monthly_sales = monthly_sales.reindex(month_order)

# Convert to Crores
monthly_sales_crore = monthly_sales / 10000000

plt.figure(figsize=(10,5))

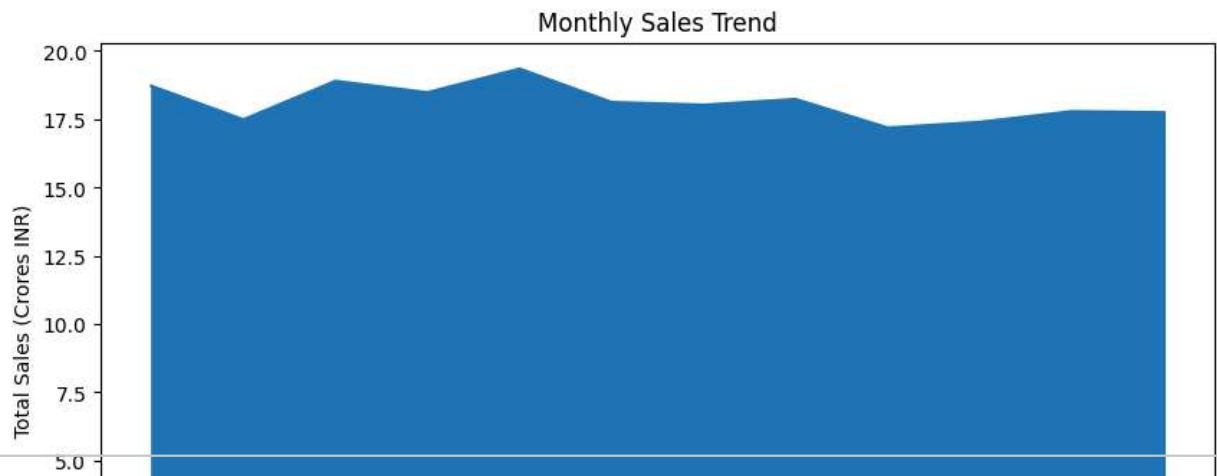
monthly_sales_crore.plot(
    kind="area"
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales (Crores INR)")

plt.xticks(rotation=45)

plt.show()
```

Date column found: ['Order_Date']



Heatmap – Correlation Analysis

```
plt.figure(figsize=(7,5))
```

```
sns.heatmap(
    final_data[
        ["Quantity","Unit_price","Total_Sales","customer_rating"]
    ].corr(),
    annot=True
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

